

Historias técnicas XP

Definición y propósito

Las historias técnicas son historias de usuario cuyo beneficiario directo es el equipo de desarrollo, no el usuario final. Abordan la infraestructura técnica, la calidad interna del código y la capacidad del equipo para seguir entregando con velocidad sostenible.

En XP, las historias técnicas tienen el mismo estatus que las historias funcionales:

- Tienen estimación en puntos de historia.
- Tienen criterios de aceptación (DoD) verificables.
- Se incluyen en el Sprint Backlog y se presentan en el Sprint Review.
- Forman parte de la velocidad del sprint.

No son tareas informales que los developers hacen “cuando tienen tiempo”. Son trabajo planificado, visible para el PO y priorizables junto con las historias funcionales.

Por qué no son tareas informales

El anti-patrón más común con historias técnicas es tratarlas como trabajo invisible: los developers las hacen en los márgenes del sprint sin que el PO las vea ni las priorice. El problema es que ese trabajo entonces compite silenciosamente con la velocidad funcional, sin que el equipo pueda comunicar por qué la velocidad baja.

La alternativa es la transparencia: las historias técnicas son ítems explícitos del backlog. El PO puede decidir priorizar una historia técnica sobre una funcional cuando la deuda técnica acumulada amenaza la velocidad del próximo sprint. Sin visibilidad, esa decisión no puede tomarse de forma informada.

En este proyecto, el PO (Josué Say) tiene visibilidad sobre las siete historias técnicas. La decisión de incluirlas en cada sprint es parte del Sprint Planning normal.

Regla del 20 %

Aproximadamente el 20 % de la capacidad de cada sprint debe reservarse para historias técnicas. Con tres developers senior a 1.0 FTE y sprints de dos semanas, esto equivale a tiempo real de desarrollo destinado a refactoring, pruebas y configuración de infraestructura técnica.

Esta reserva no es un techo: es un piso. Cuando la deuda técnica acumulada o la complejidad de una épica lo requieren (como en los sprints 5-7 que concentran EP04, EP05 y EP06), la proporción puede ser mayor.

La regla del 20 % evita el anti-patrón de relegación: que las historias técnicas se posterguen sprint tras sprint bajo la presión de completar funcionalidad, hasta que la deuda técnica acumulada obliga a una pausa no planificada.

Las siete historias técnicas del proyecto

HT-01: Configurar pipeline CI

Campo	Detalle
Sprint	1
Práctica XP	Continuous Integration
Estimación	A definir en Sprint Planning (prioridad alta)

Descripción. Configurar el pipeline de integración continua desde el primer sprint. El pipeline debe ejecutar compilación, pruebas unitarias y reporte de cobertura de forma automática en cada push al repositorio principal.

Criterios de aceptación.

- El pipeline ejecuta sin errores en el repositorio del proyecto.
- Cualquier push al repositorio principal activa el pipeline automáticamente.
- El pipeline reporta estado verde cuando todas las pruebas pasan y el build compila.
- El pipeline reporta estado rojo con información de diagnóstico cuando algo falla.

Por qué en sprint 1. Configurar CI tardíamente es un anti-patrón explícito. Un pipeline activo desde el sprint 1 protege todos los sprints siguientes. Un pipeline configurado en el sprint 5 no ha protegido los cuatro sprints anteriores.

HT-02: Definir y documentar estándares de codificación

Campo	Detalle
Sprint	1
Práctica XP	Collective Ownership
Estimación	A definir en Sprint Planning

Descripción. Acordar y documentar los estándares de codificación del proyecto antes de que comience el desarrollo de funcionalidad. Los estándares incluyen convenciones de nomenclatura, estructura de archivos, manejo de errores, formato de código y herramientas de linting.

Criterios de aceptación.

- El documento de estándares está disponible en el repositorio y accesible para todos los members del equipo.

- Las herramientas de linting configuradas en el pipeline CI verifican los estándares automáticamente.
- Los tres developers han revisado y aceptado los estándares antes del primer commit de funcionalidad.

Por qué en sprint 1. Collective Ownership requiere que cualquier developer pueda modificar cualquier parte del código sin ambigüedad sobre convenciones. Los estándares de codificación son la condición necesaria para que Collective Ownership funcione en la práctica.

HT-03: Suite de pruebas unitarias EP02

Campo	Detalle
Sprint	2
Práctica XP	TDD
Estimación	A definir en Sprint Planning

Descripción. Implementar la suite de pruebas unitarias para la lógica de guardado automático de EP02 (encuesta vocacional). Cubre los casos de estado del progreso de la encuesta, guardado parcial, recuperación de sesión interrumpida y validación de respuestas incompletas.

Criterios de aceptación.

- Las pruebas cubren los casos principales de la lógica de guardado automático.
- Las pruebas se ejecutan en verde en el pipeline CI.
- Cada caso de prueba corresponde a un criterio de aceptación de las historias funcionales de EP02.

HT-04: Refactorización módulo autenticación y registro EP01

Campo	Detalle
Sprint	3
Práctica XP	Refactoring
Estimación	A definir en Sprint Planning

Descripción. Refactorizar el módulo de autenticación y registro de usuarios (EP01) después de la implementación inicial en sprints 1-2. El objetivo es mejorar la estructura interna, eliminar duplicaciones identificadas durante el desarrollo y asegurar que el código cumple los estándares definidos en HT-02.

Criterios de aceptación.

- El comportamiento externo del módulo no cambia: las pruebas existentes siguen en verde.
- La deuda técnica identificada durante los sprints 1-2 está resuelta y documentada como resuelta.
- El código resultante cumple los estándares de codificación definidos en HT-02.
- No se introducen nuevas funcionalidades en esta historia.

Por qué en sprint 3. El sprint 3 es el primer punto donde EP01 ha madurado lo suficiente para que el refactoring sea informado por la experiencia real de implementación, y antes de que los módulos de sprints posteriores (EP03, EP04) dependan del módulo de EP01 de forma intensiva.

HT-05: Pruebas unitarias motor de recomendación EP04

Campo	Detalle
Sprint	5
Práctica XP	TDD
Estimación	A definir en Sprint Planning (prioridad alta, alto esfuerzo)

Descripción. Implementar la suite de pruebas unitarias para el motor de recomendación (EP04). Cubre el algoritmo de scoring por afinidad, el ranking de resultados, el determinismo de resultados para perfiles iguales y los casos límite del modelo de recomendación.

Criterios de aceptación.

- Las pruebas cubren el algoritmo de scoring de EP04-US01 (8 puntos de historia).
- Las pruebas verifican el determinismo: para el mismo perfil vocacional, el ranking producido es siempre el mismo.
- Los casos límite del modelo están probados: perfiles con afinidades iguales entre múltiples carreras, perfiles con respuestas incompletas, carreras sin correspondencia clara.
- Las pruebas se ejecutan en verde en el pipeline CI.

Por qué en sprint 5. EP04 es el componente de mayor criticidad técnica del proyecto. Sin pruebas unitarias sólidas, no hay forma confiable de verificar que el motor produce resultados correctos o de hacer refactoring posterior sin riesgo de regresión.

HT-06: Refactorización motor de recomendación post-implementación

Campo	Detalle
Sprint	6
Práctica XP	Refactoring
Estimación	A definir en Sprint Planning

Descripción. Refactorizar el motor de recomendación (EP04) después de la implementación del sprint 5. El objetivo es mejorar la legibilidad, eliminar duplicaciones y asegurar que el código puede ser comprendido y modificado por cualquier developer del equipo (Collective Ownership).

Criterios de aceptación.

- El comportamiento externo del motor no cambia: la suite de pruebas de HT-05 sigue en verde.
- El código es comprensible para un developer que no implementó el motor originalmente.
- La deuda técnica identificada durante el sprint 5 está resuelta.
- No se introducen nuevas funcionalidades de EP05 en esta historia.

Por qué en sprint 6. El sprint 6 inicia el desarrollo de EP05 (clasificación de afinidad y ponderación), que construye sobre EP04. Refactorizar EP04 al inicio del sprint 6 —antes de que EP05 añada complejidad adicional— es el momento óptimo.

HT-07: Revisión de seguridad manejo datos personales EP01/EP02

Campo	Detalle
Sprint	4
Práctica XP	Simple Design, Continuous Integration
Estimación	A definir en Sprint Planning

Descripción. Revisar el manejo de datos personales de estudiantes en EP01 (registro y perfil) y EP02 (encuesta vocacional) para verificar que la implementación cumple los principios de Simple Design en seguridad: no se almacena más información de la necesaria, los datos sensibles se protegen apropiadamente y el manejo de errores no expone información interna del sistema.

Criterios de aceptación.

- Se revisa el conjunto de datos personales almacenados y se verifica que es el mínimo necesario.
- El manejo de errores en EP01 y EP02 no expone detalles técnicos del sistema al usuario final.
- Las pruebas de regresión en CI incluyen casos que verifican que datos sensibles no se exponen en respuestas de error.

- Los hallazgos de la revisión están documentados en el backlog (como ítems resueltos o como nuevas historias si se identifican problemas).

Por qué en sprint 4. El sprint 4 completa EP03 (catálogo de carreras) y es el punto antes de EP04 donde el equipo tiene capacidad para revisar los módulos de EP01/EP02 ya implementados. R8 (exposición de datos personales, probabilidad 0.20) justifica esta revisión explícita.

Resumen de historias técnicas

ID	Descripción	Sprint	Práctica XP
HT-01	Configurar pipeline CI	1	Continuous Integration
HT-02	Estándares de codificación	1	Collective Ownership
HT-03	Suite de pruebas EP02	2	TDD
HT-04	Refactorización EP01	3	Refactoring
HT-05	Pruebas unitarias EP04	5	TDD
HT-06	Refactorización EP04	6	Refactoring
HT-07	Revisión seguridad EP01/EP02	4	Simple Design, CI

Anti-patrón: historias técnicas invisibles para el PO

El anti-patrón más dañino con historias técnicas es mantenerlas fuera del Product Backlog y del Sprint Planning, bajo el argumento de que “son cosas técnicas que el PO no necesita ver”.

Las consecuencias son directas:

- Las historias técnicas compiten silenciosamente con la capacidad del sprint sin que el PO pueda priorizar entre ellas y las historias funcionales.
- Cuando la deuda técnica acumulada finalmente impacta la velocidad, el PO no tiene información para entender por qué el equipo entrega menos de lo esperado.
- Las historias técnicas se postergan indefinidamente hasta que se convierten en problemas urgentes que consumen más capacidad de la que habrían consumido si se hubieran planificado.

En este proyecto, el PO tiene visibilidad completa sobre HT-01 a HT-07. La priorización de estas historias es una decisión explícita de cada Sprint Planning, no trabajo invisible del equipo.